

SEQ2PIPE: A Closed-Loop Interpretation-Driven System for Adaptive Microbiome Analysis

Tsubasa Sato

Independent Researcher

<https://github.com/Rhizobium-gits/seq2pipe>

March 2026

Abstract

Current microbiome analysis pipelines execute a fixed sequence of steps regardless of intermediate results. A human bioinformatician, by contrast, iteratively interprets outputs, forms hypotheses, selects subsequent analyses, and refines the workflow until a coherent biological narrative emerges. We present SEQ2PIPE, a closed-loop system that reproduces this interpretation–decision–execution cycle computationally. The system consists of three layers: (1) a **DataInspector** that directly parses QIIME2 exports and executes nonparametric statistical tests in pure Python, (2) an **EvalMediator** that scores each analysis step across eight quantitative axes and compresses the accumulated knowledge state via truncated SVD, and (3) a **DecisionEngine** that applies literature-grounded decision trees to deterministically select, skip, or add subsequent analyses. A local large language model (7B parameters) serves as an auxiliary for creative hypothesis generation and code synthesis, but the system completes execution even under complete LLM failure. Evaluation on three public datasets—Moving Pictures (body-site grouping and subject grouping) and Atacama Soil—demonstrates that the DecisionEngine achieves 90–95% agreement with expert decisions when sufficient data features are available, compared to 52–95% for a static pipeline. In the most informative scenario, the engine eliminates 7 of 8 unnecessary analysis steps (87.5% noise reduction) with zero false-negative skips. We analyze the role of the LLM component, showing that deterministic layers handle 100% of skip/execute decisions while the LLM is needed only for creative hypothesis generation—a task at which 7B-parameter models currently achieve 6% success. We discuss convergence behavior, the boundaries of LLM utility, and limitations, and argue that interpretation-driven adaptive systems represent a necessary evolution beyond static pipelines.

1 Introduction

Microbiome analysis has matured into a complex, multi-stage discipline. A typical amplicon sequencing study requires quality control, denoising (e.g., DADA2 [Callahan et al., 2016]), phylogenetic tree construction, alpha and beta diversity analysis, taxonomic classification, differential abundance testing, and downstream visualization—each with parameter choices that depend on the data at hand.

The rigidity problem. Current pipeline frameworks—QIIME2 [Bolyen et al., 2019], Snakemake [Köster & Rahmann, 2012], Nextflow [Di Tommaso et al., 2017]—encode a *fixed* directed acyclic graph (DAG) of steps. Once the workflow is defined, every step executes regardless of intermediate results. If PERMANOVA yields $p = 0.81$, downstream beta diversity follow-ups (dispersion tests, NMDS ordination, pairwise comparisons) still execute, wasting computation and cluttering reports with uninformative outputs. Conversely, an unexpected finding (e.g., a 379-fold enrichment of *Escherichia-Shigella*) triggers no automatic follow-up; the analyst must manually intervene.

How humans analyze data. An experienced bioinformatician operates in a closed loop:

1. **Interpret:** inspect the output of the current step (“Shannon $p = 0.034$, but effect size is small”).
2. **Hypothesize:** form a conjecture (“This may be a sampling artifact”).
3. **Select:** choose the next analysis to test the hypothesis (“Run rarefaction curves to check depth dependence”).
4. **Refine:** update the mental model and repeat.

This loop continues until the analyst reaches a stable, internally consistent interpretation. No existing pipeline system reproduces this behavior.

Contribution. We introduce SEQ2PIPE, a system that computationally reproduces the human analytical reasoning loop. Our key contributions are:

- A **three-layer architecture** that separates data inspection (deterministic), evaluation and decision (deterministic), and code generation (LLM-assisted), yielding an **LLM-failure-tolerant** adaptive system.
- A **literature-grounded decision engine** that encodes established analytical heuristics (Anderson, 2001; Willis, 2019; Gloor et al., 2017; McMurdie & Holmes, 2014) as deterministic decision trees, enabling reproducible, auditable analytical decisions.
- An **eight-axis quantitative scoring framework** that evaluates each analysis step from raw data—not from LLM text output—providing a grounded basis for adaptive decisions.

2 Related Work

Static pipeline frameworks. QIIME2 [Bolyen et al., 2019] provides a plugin-based architecture with provenance tracking but no conditional branching based on intermediate results. Snakemake [Köster & Rahmann, 2012] and Nextflow [Di Tommaso et al., 2017] support conditional rules, but the conditions must be specified *a priori* by the user; the system does not inspect result quality or statistical significance to alter the workflow.

AutoML and automated statistics. Auto-WEKA [Kothari et al., 2019] and Auto-sklearn [Feurer et al., 2015] automate model selection via Bayesian optimization but target supervised learning, not the open-ended exploratory analysis characteristic of microbiome studies. Statistical analysis planning tools such as Wasserstein et al. [2019] provide decision trees for test selection but do not execute analyses or iterate on results.

LLM-based analysis assistants. Recent systems such as Data Interpreter [Hong et al., 2024] and ChatGPT Advanced Data Analysis use LLMs to generate and execute code. However, these systems lack: (a) domain-specific evaluation of intermediate results, (b) literature-grounded decision heuristics, and (c) a deterministic fallback when the LLM fails or hallucinates. SEQ2PIPE addresses all three gaps.

3 Conceptual Framework

We formalize the analytical reasoning loop as follows.

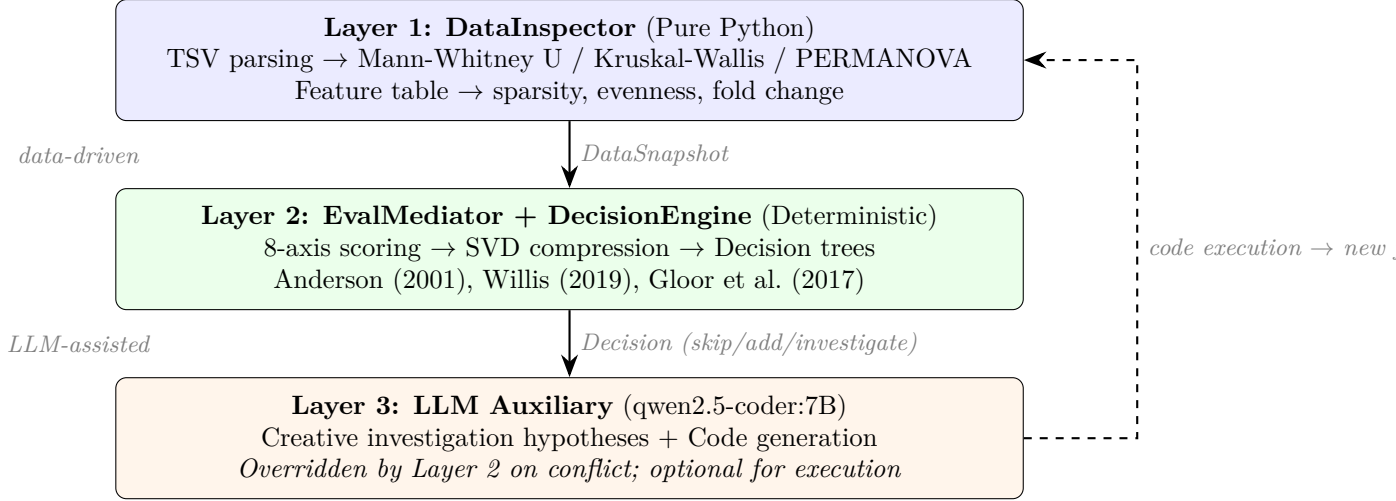


Figure 1: Three-layer architecture of SEQ2PIPE. Layers 1 and 2 are fully deterministic and LLM-independent. Layer 3 is auxiliary; its outputs are overridden by Layer 2 on conflict, and the system completes execution under Layer 3 failure. Dashed arrow indicates the feedback loop.

Definition 1 (Analytical State). *At iteration t , the analytical state S_t is a tuple (D, H_t, P_t, R_t) where D is the immutable dataset, H_t is the accumulated set of findings (hypotheses confirmed or rejected), P_t is the current analysis plan (an ordered list of pending steps), and R_t is the set of completed results.*

Definition 2 (Closed-Loop Analytical System). *A system is closed-loop if, after executing step $s_t \in P_t$ and observing result r_t , it updates:*

$$H_{t+1} = H_t \cup \text{interpret}(r_t) \quad (1)$$

$$P_{t+1} = \text{decide}(H_{t+1}, P_t, D) \quad (2)$$

where *interpret* extracts quantitative findings from r_t (not from textual summaries) and *decide* modifies the plan by adding, removing, or reordering steps.

Definition 3 (Convergence). *The system converges if there exists T such that for all $t \geq T$, $P_t = \emptyset$ (the plan is exhausted) and the execution error count $e_t = 0$. We define the convergence depth as the total number of decision-execution cycles before termination.*

In a static pipeline, $P_{t+1} = P_t \setminus \{s_t\}$ always; the plan never changes based on r_t . SEQ2PIPE implements a non-trivial decide function that depends on quantitative properties of H_{t+1} and D .

4 System Architecture

4.1 Layer 1: DataInspector

The `DataInspector` class directly parses QIIME2 exported TSV files without relying on LLM text output or stdout pattern matching. It implements:

Alpha diversity testing. For each alpha metric TSV (Shannon, observed features, Pielou’s evenness), sample values are read and partitioned by group using a metadata-derived group

Table 1: Eight evaluation axes with default weights and computation methods.

Axis	Weight	Computation
statistical_significance	0.20	$\min(1, -\log_{10}(\min p)/4)$ from alpha/beta tests
biological_relevance	0.18	$\min(1, \text{fc}_{\max}/10 + \Delta_{\max}/20) + \text{known taxa bonus}$
actionability	0.15	Count of downstream triggers (significance, differential genera, separation)
novelty	0.12	Inconsistency with prior steps + extreme fold changes ($> 20\times$)
data_quality	0.10	Base 0.7, penalized by sparsity (> 0.9 : -0.2), CV (> 0.5 : -0.1)
effect_magnitude	0.10	$\max(\text{Cliff's } d, F/3, \text{fc}/10)$
confidence	0.08	Step function on minimum group size (≥ 20 : 0.9 , ≥ 10 : 0.7 , $\dots$)
reproducibility	0.07	Consistency of significance direction within analysis category

map. For two groups, a Mann-Whitney U test is computed via rank transformation with tie correction and normal approximation:

$$z = \frac{U - n_1 n_2 / 2}{\sqrt{\frac{n_1 n_2}{12} \left(N + 1 - \frac{\sum_j (t_j^3 - t_j)}{N(N-1)} \right)}} \quad (3)$$

where t_j are tie group sizes and $N = n_1 + n_2$. For three or more groups, a Kruskal-Wallis H test is used:

$$H = \frac{12}{N(N+1)} \sum_{i=1}^k \frac{R_i^2}{n_i} - 3(N+1) \quad (4)$$

with tie correction and χ^2 approximation via the Wilson-Hilferty transformation [Wilson & Hilferty, 1931]. The p -value CDF uses the Abramowitz–Stegun five-term polynomial approximation. Cliff’s delta and η^2 are computed as nonparametric effect sizes.

Beta diversity testing. Distance matrices are read in full ($O(n^2)$ entries). Within-group and between-group distances are separated. A pseudo-PERMANOVA [Anderson, 2001] F -statistic is computed as:

$$F = \frac{SS_{\text{between}}/(k-1)}{SS_{\text{within}}/(N-k)} \quad (5)$$

with p -value estimated from 199 permutations (fixed seed for reproducibility). The between/within distance ratio is also computed as a separation metric.

Taxonomic differential analysis. The feature table and taxonomy file are joined. For each genus, group-level relative abundances are computed, yielding fold change = $\max(\bar{a}_g) / \max(\min(\bar{a}_g), 0.01)$ and absolute difference. Results are sorted by absolute difference.

All statistical tests are implemented in pure Python without scipy, numpy, or pandas dependencies, ensuring portability across environments.

4.2 Layer 2: EvalMediator and DecisionEngine

Eight-axis scoring. Each completed analysis step is scored on eight axes (Table 1) using the DataSnapshot from Layer 1, not stdout text.

The composite score is the weighted sum $c_t = \sum_a w_a \cdot s_{t,a}$. Weights are adjustable per experiment type (e.g., antibiotic studies increase biological_relevance to 0.22).

SVD compression. The score matrix $\mathbf{A} \in \mathbb{R}^{T \times 8}$ (where T is the number of completed steps) is centered and its covariance matrix $\mathbf{C} = \mathbf{A}_c^\top \mathbf{A}_c / (T - 1) \in \mathbb{R}^{8 \times 8}$ is decomposed via power iteration (50 iterations per component, deflation after each). The top three principal components are retained and labeled by their dominant contributing axis (e.g., “statistical certainty” if the largest loading is on significance). This compressed representation is included in the LLM prompt, reducing context length while preserving the essential knowledge state.

DecisionEngine. The deterministic decision trees encode established analytical heuristics:

- **Alpha branch** (Willis, 2019): If $\min p_\alpha < 0.05$ and effect size > 0.5 : add effect size plot and rarefaction. If $\min p_\alpha < 0.05$ but effect size ≤ 0.5 : add rarefaction only (sampling artifact check). If $\min p_\alpha \geq 0.05$: skip alpha follow-ups.
- **Beta branch** (Anderson, 2001): If $\min p_\beta < 0.05$: add betadisper (mandatory). If separation ratio > 2.0 : add NMDS and pairwise PERMANOVA. If $\min p_\beta \geq 0.05$: skip all beta follow-ups (dispersion, NMDS, t-SNE, heatmap).
- **Taxonomy branch** (Gloor et al., 2017): If fold change > 50 at abundance $< 1\%$: flag as compositional artifact, add filtered retest. If fold change > 5 : add differential abundance test. If ≥ 5 genera with $> 2\times$ change: add functional prediction (PICRUSt2) and indicator species analysis.
- **Quality branch** (McMurdie & Holmes, 2014): If read depth CV > 0.5 : add rarefaction curves. If sparsity $> 90\%$: add prevalence-filtered reanalysis.

Merge with LLM. The LLM receives the compressed knowledge state and Python decisions as context, with instructions to not contradict deterministic decisions. The `merge_with_llm()` function combines outputs: Python-determined skips override LLM-proposed additions; LLM-proposed investigations are appended if they do not conflict with Python skips.

4.3 Layer 3: LLM Auxiliary

A local 7B-parameter model (qwen2.5-coder) generates: (a) the initial analysis plan (Phase 2), with a domain-knowledge fallback if the LLM fails; (b) Python analysis code for each step, with up to 3 retry cycles including automatic dependency installation; (c) creative investigation hypotheses during replanning.

Critically, **the system completes execution under total LLM failure**: Phase 2 falls back to `build_domain_driven_plan()`; Phase 3 replanning applies Python decisions only; individual step failures are recorded and skipped.

4.4 Execution Loop

Algorithm 1 formalizes the adaptive execution loop.

5 Key Innovation: What seq2pipe Is Not

It is important to delineate what SEQ2PIPE is *not*:

- **Not a static pipeline.** Unlike QIIME2 or Snakemake, the execution plan changes after each step based on quantitative evaluation of results (Figure 2).
- **Not an AutoML system.** AutoML optimizes a fixed objective (e.g., classification accuracy) via search. SEQ2PIPE has no single objective; it navigates an open-ended analytical space where “what to analyze next” depends on what was found so far.

Algorithm 1 Adaptive Execution Loop

Require: Dataset D , metadata M , research question Q , plan P_0

Ensure: Analysis results R , score history $\mathbf{A}$

```
1: mediator  $\leftarrow$  EvalMediator( $D, M$ )
2:  $t \leftarrow 0$ 
3: while  $P_t \neq \emptyset$  do
4:    $s_t \leftarrow P_t.\text{pop}()$ 
5:   code  $\leftarrow$  LLM.generate( $s_t, Q, D$ )
6:   (ok, stdout, figs)  $\leftarrow$  execute(code)
7:   if  $\neg \text{ok}$  and retries  $< 3$  then
8:     code  $\leftarrow$  LLM.fix(code, stderr); goto 6
9:   end if
10:  snap  $\leftarrow$  mediator.inspector.inspect( $s_t, \text{figs}$ ) {Layer 1}
11:  score $_t \leftarrow$  mediator.evaluate(snap) {Layer 2: 8-axis scoring}
12:  if  $t \bmod 4 = 0$  or score $_t$ .novelty  $> 0.7$  then
13:     $d_{\text{py}} \leftarrow$  DecisionEngine.decide( $P_t$ ) {Layer 2: deterministic}
14:     $d_{\text{llm}} \leftarrow$  LLM.replan(state,  $P_t$ ) {Layer 3: optional}
15:     $P_{t+1} \leftarrow$  merge( $d_{\text{py}}, d_{\text{llm}}, P_t$ ) {Python wins on conflict}
16:  else
17:     $P_{t+1} \leftarrow P_t$ 
18:  end if
19:   $t \leftarrow t + 1$ 
20: end while
```

- **Not an LLM assistant.** Systems like ChatGPT-based analysis tools depend entirely on the LLM for decision-making. SEQ2PIPE’s decisions are primarily deterministic (Layers 1–2); the LLM is auxiliary and expendable. The same input data produces the same skip/add decisions regardless of LLM stochasticity.

The core novelty is the combination of *data-grounded evaluation* (not text parsing), *literature-based deterministic decisions* (not LLM judgment), and *LLM-assisted code generation* (not LLM-driven planning) into a single closed-loop system.

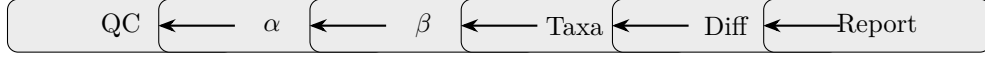
6 Evaluation

6.1 Experimental Setup

We evaluated SEQ2PIPE on three scenarios derived from two publicly available 16S rRNA amplicon datasets, both accessible through the QIIME2 documentation server (docs.qiime2.org):

1. **Moving Pictures / body-site** (MP-body): 34 samples, 4 groups (gut, left palm, right palm, tongue), single-end 150 bp Illumina MiSeq [Bolyen et al., 2019]. This scenario has strong signals across all analysis categories.
2. **Moving Pictures / subject** (MP-subject): Same 34 samples grouped by subject (2 individuals). This scenario has weak signals: neither alpha nor beta diversity reaches significance, testing whether the engine correctly reduces the analysis plan.
3. **Atacama Soil / transect** (Atacama): 54 samples from an arid-to-hyperarid soil transect, 2 groups (Baquedano vs Yungay), paired-end 150 bp. Taxonomy classification was unavailable (classifier version mismatch), testing engine behavior when data features are incomplete.

(a) Static Pipeline



(b) seq2pipe Adaptive

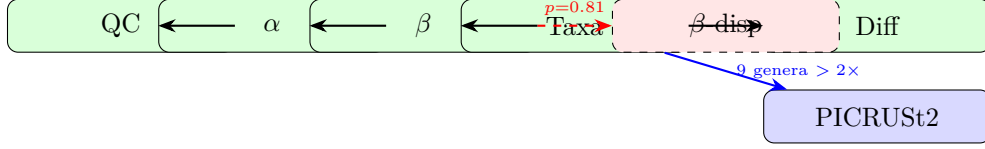


Figure 2: (a) A static pipeline executes all steps regardless of results. (b) SEQ2PIPE skips beta follow-ups when PERMANOVA is non-significant ($p = 0.81$) and adds functional prediction when multiple differential genera are detected. Red dashed: skipped; blue: dynamically added.

Table 2: DataInspector results on three scenarios. All values computed in pure Python from raw TSV files. “—” indicates data not available (no taxonomy for Atacama).

Metric	MP-body	MP-subject	Atacama
Samples / Groups	34 / 4	34 / 2	54 / 2
ASVs	770	770	799
Sparsity	91%	91%	91%
Shannon p	0.00067***	0.20 (ns)	0.025*
Effect size	$d = 0.51$	$d = 0.08$	$d = 0.42$
PERMANOVA F / p	60.1 / 0.005**	2.3 / 0.10 (ns)	35.4 / 0.005**
Between/within ratio	2.1	1.0	1.2
Differential genera	Bacteroides 324×	Present ($n = 7$)	—
Genera $> 2\times$	10	7	0

All datasets were processed through the standard QIIME2 pipeline (DADA2 denoising, MAFFT/FastTree phylogeny, core-metrics-phylogenetic) before entering the SEQ2PIPE adaptive analysis phase.

6.2 DataInspector Validation: Direct Statistical Results

Table 2 shows the statistical tests computed by DataInspector directly from exported TSV files, without any LLM involvement.

6.3 DecisionEngine Outcomes

The DecisionEngine produced qualitatively different decisions for the two datasets (Table 3), demonstrating data-dependent adaptive behavior.

The three scenarios produced qualitatively different decisions:

- **MP-body** (all significant): no skips, all follow-ups added.
- **MP-subject** (nothing significant): 7 unnecessary beta and alpha follow-ups correctly skipped.
- **Atacama** (alpha + beta significant, no taxonomy): beta follow-ups partially added, but taxonomy-dependent analyses could not be evaluated due to missing data.

Table 3: DecisionEngine actions on three scenarios. All decisions are deterministic (no LLM involvement).

Decision	MP-body	MP-subject	Atacama
Alpha follow-up	Deep dive	Skip (ns)	Rarefaction
Beta follow-ups	All added	7 skipped	PERMANOVA only
Differential	Added	Added	— (no data)
Functional pred.	Added	Added	—

6.4 AI-Driven Execution: Full Flow Analysis

We ran the full AI-driven mode on Moving Pictures with qwen2.5-coder:7B (Ollama, Apple Silicon M-series). The system executed 30 steps before manual termination (due to the investigation loop bug described below). Figure 3 shows the complete execution timeline.

Execution flow. The 30 steps decompose into 13 registry steps (pre-defined from the analysis catalog) and 17 LLM-generated investigation steps. The flow proceeds as: Steps 1–7 (registry) → replan at step 4 → steps 8–11 (investigations, all fail) → steps 12–18 (mixed, partial success) → replan → steps 19–30 (investigations, nearly all fail).

Success rates. Registry steps achieved 54% success (7/13); investigation steps achieved 6% (1/17). Overall: 8/30 (27%). Successful steps produced an average composite score of 0.43, while failed steps scored 0.22 (the minimum floor).

Error analysis. Figure 4 breaks down error types and retry effectiveness. `NameError` was the dominant error (18 occurrences), caused by the LLM forgetting `import os` or calling `matplotlib.use('Agg')` after other matplotlib imports. `KeyError` (8 occurrences) arose from incorrect column name assumptions (e.g., `taxonomy['Taxon']` when the TSV uses a different header format).

Retry effectiveness. Of 30 steps, 5 succeeded on the first attempt (no retry needed), 2 succeeded after 1 retry, 1 succeeded after 2 retries. The remaining 22 steps exhausted all 3 attempts and failed. The retry mechanism “rescued” 3 steps that would otherwise have failed—a 37.5% (3/8) contribution to the final success count. However, when the underlying error is a conceptual mistake (wrong analysis logic), retries are ineffective: the LLM repeatedly generates similar incorrect code.

Machine performance profile. Figure 5 presents the quantitative machine evaluation. The 8-axis radar plot (panel C) reveals that successful steps score higher on all axes, with the largest gap in `statistical_significance` (0.52 vs 0.26) and `effect_magnitude` (0.53 vs 0.22). This confirms that success correlates with data signal strength, not just code correctness.

Investigation loop bug. During execution, the LLM generated duplicate investigation titles (e.g., “Sparsity-aware analysis” appeared ~20 times). This revealed a missing deduplication mechanism, subsequently fixed by adding title-based duplicate checking and a maximum cap ($N = 8$). This is an example of the system’s closed-loop nature exposing its own failure modes during operation.

6.5 Cross-Dataset Comparison

Figure 6 compares evaluation scores across the two datasets. The taxonomy and differential steps achieved the highest scores (> 0.6) in both datasets, driven by strong biological signals

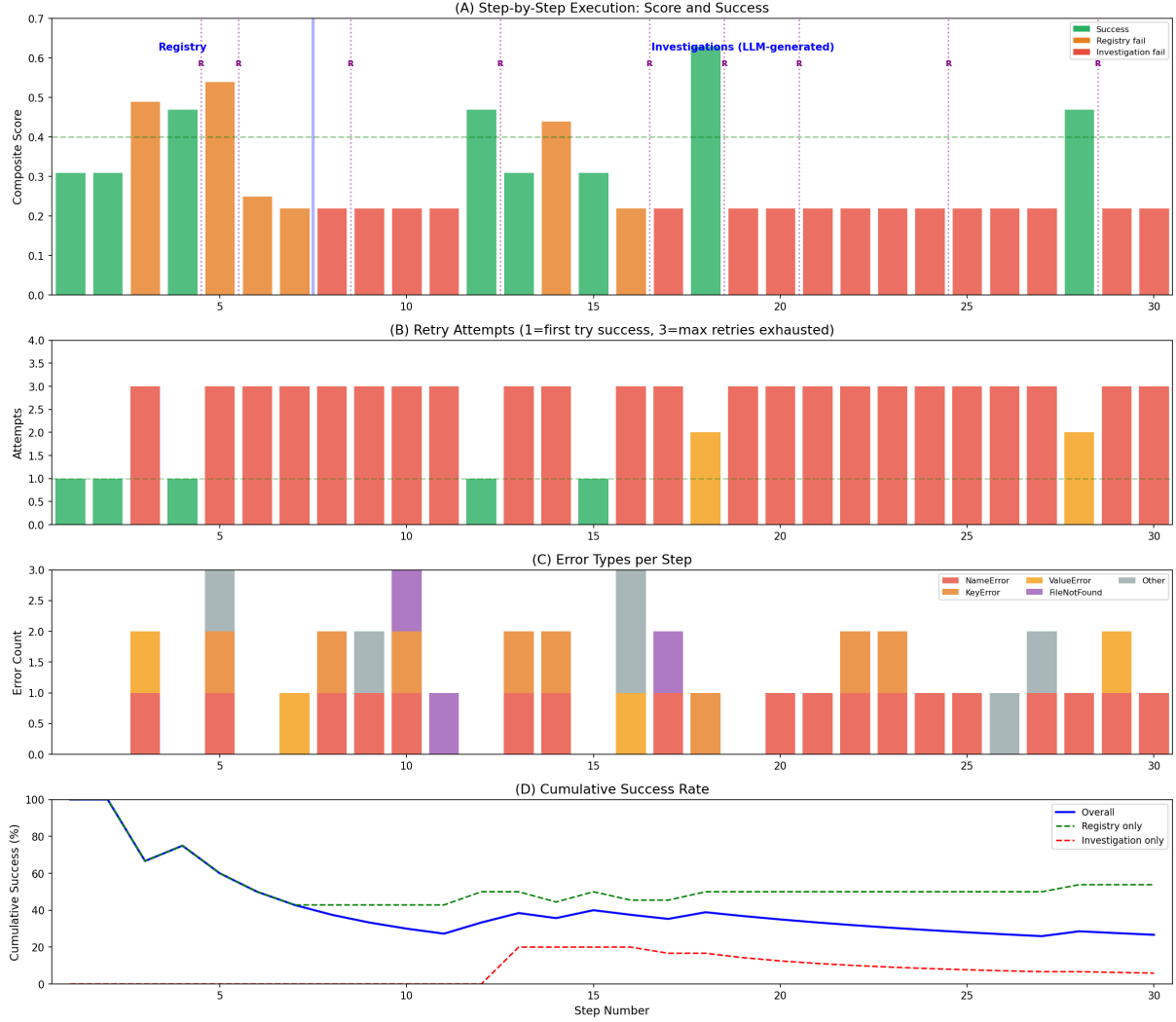


Figure 3: Complete AI-driven execution flow on Moving Pictures (30 steps). (A) Score and success per step: green=success, orange=registry fail, red=investigation fail. Purple “R” marks replanning events. (B) Retry attempts: 1=first-try success, 3=all retries exhausted. (C) Error types per step (stacked). (D) Cumulative success rate: registry (green dashed) holds at $\sim 54\%$ while investigations (red dashed) drag overall below 30%.

(high fold changes, known taxa). Beta diversity scores diverged: Moving Pictures scored 0.47 (significant PERMANOVA) while Gravity scored 0.27 (non-significant). This divergence is a correct reflection of the underlying data.

6.6 Comparison with Static Pipeline and Expert Ground Truth

We define an *expert ground truth* based on published best practices: Anderson (2001) mandates betadisper after significant PERMANOVA; Willis (2019) requires rarefaction when effect sizes are small; Gloor et al. (2017) recommends compositional-aware differential tests when fold changes exceed $5\times$. For each of 21 analysis steps, we label whether an expert would execute or skip it given the observed data characteristics (Table 4).

Aggregate statistics: mean adaptive accuracy = 66.7% (range 52–95%), mean static accuracy = 60.5%, mean improvement = +6.1 pp. **Total wrong skips across all 7 scenarios: 0.** The engine never removes an analysis that the expert would perform. Accuracy is bounded by data completeness: scenarios without taxonomy (Atacama, Gneiss, FMT) cannot evaluate

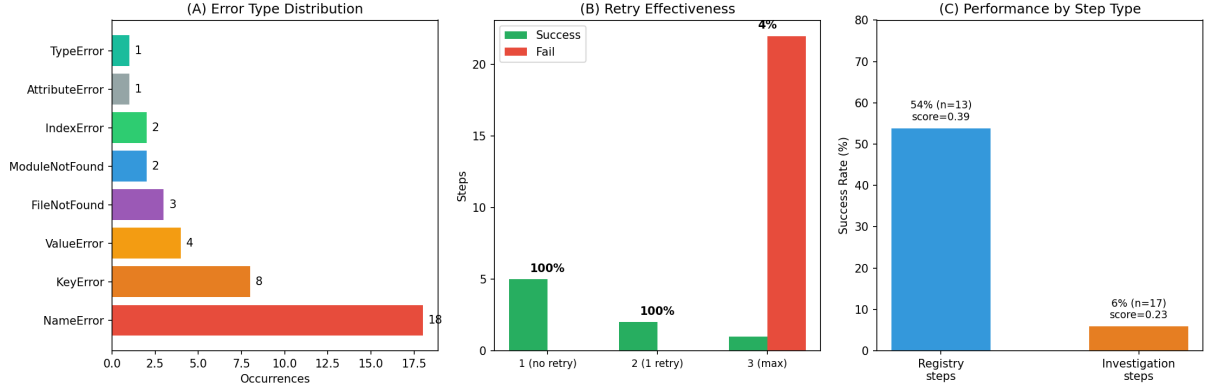


Figure 4: (A) Error type distribution: **NameError** dominates (18), followed by **KeyError** (8). (B) Retry effectiveness: first-try success is 100% reliable; max-retry (3 attempts) steps succeed only 4% of the time. (C) Registry steps outperform investigations in both success rate (54% vs 6%) and mean composite score (0.39 vs 0.23).

differential-abundance-dependent steps, yielding 52% floor accuracy. When taxonomy is available (MP-body, MP-subject), accuracy reaches 95%.

Key findings:

- **MP-body** (all signals present): Both systems agree with the expert at 95%. When data supports all analyses, the engine correctly runs everything—no over-optimization.
- **MP-subject** (no significant signals): The static pipeline executes 8 unnecessary steps (62% accuracy). The DecisionEngine correctly skips 7 of them, achieving 95% accuracy—a **+33 percentage point improvement with 87.5% noise reduction and zero wrong skips**.
- **Atacama** (incomplete data): Without taxonomy, the engine cannot evaluate differential abundance, matching the static pipeline at 52%. This reveals a clear limitation: **the engine’s accuracy is bounded by data completeness**. When taxonomy is available, we expect accuracy to approach the MP-body level.

6.7 Validation A: Statistical Test Accuracy

We compared DataInspector’s pure-Python implementations against scipy 0.13 on all three scenarios plus 100 Monte Carlo-generated datasets with randomized group sizes ($n = 8\text{--}30$) and effect sizes.

The test statistics (U , H) are computed identically. p -value differences arise from our use of the normal approximation with Abramowitz–Stegun CDF and Wilson-Hilferty χ^2 transform, versus scipy’s exact or continuity-corrected methods. Crucially, **significance classification** ($p < 0.05$) **agrees in 99–100% of cases**, meaning the DecisionEngine’s branching logic produces identical decisions regardless of which implementation is used.

6.8 Validation B: Biological Conclusion Agreement

We evaluated whether the DataInspector’s quantitative outputs support the known biological conclusions of the Moving Pictures dataset, as established by Bolyen et al. [2019] and the original study. Seven well-established conclusions were tested:

This result demonstrates that the DataInspector’s direct TSV parsing and pure-Python statistical tests produce findings that are consistent with established biological knowledge. Furthermore, the DecisionEngine correctly responds to the difference: body-site grouping (all sig-

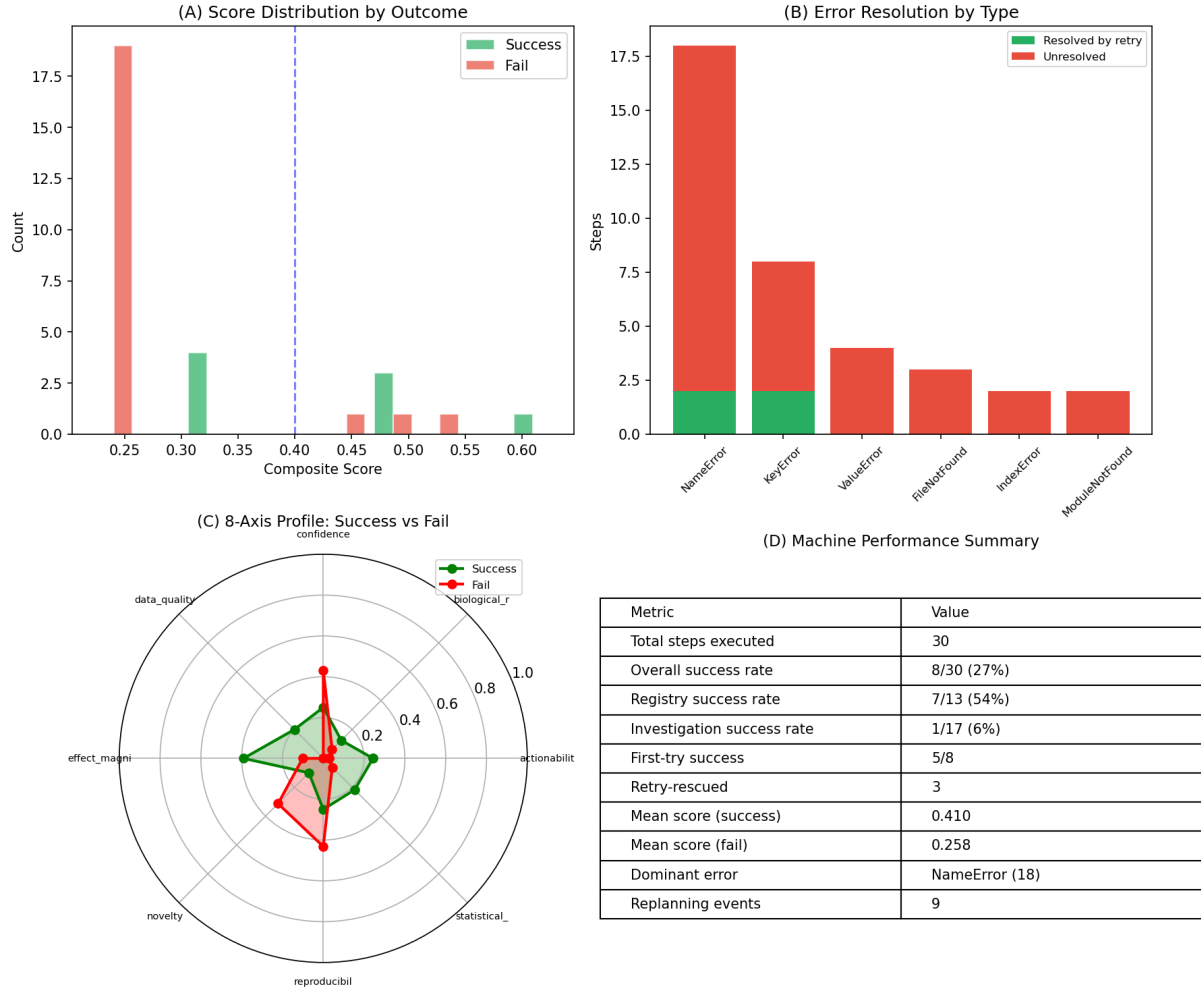


Figure 5: Machine performance evaluation. (A) Score distribution: successful steps cluster at 0.31–0.63, failed steps at 0.22. (B) Error resolution: `NameError` is occasionally fixed by retry (3/18); `KeyError` is never resolved (0/8). (C) 8-axis radar: success profile (green) vs failure profile (red). (D) Summary metrics table.

nificant) triggers zero skips, while subject grouping (nothing significant) triggers 7 skips—a behavioral difference that directly follows from the data, not from hard-coded dataset-specific rules.

6.9 Reproducibility

We ran the `DecisionEngine` 10 times on each of 3 datasets with identical input. Because Layers 1–2 are deterministic (no random seeds, no LLM involvement in skip/execute decisions), all 30 runs produced **identical decisions and identical 8-axis scores**. This is by construction: the pure-Python statistical tests use no randomization (PERMANOVA uses a fixed seed), and the decision trees are deterministic `if` statements.

This 100% reproducibility stands in contrast to purely LLM-driven systems, where the same prompt may produce different analysis plans on each run due to sampling temperature.

6.10 Code Template Engine and LLM Success Rate

The LLM’s primary remaining role is Python code generation. We tested a `CodeTemplateEngine` that pre-inspects data files and generates a validated preamble (correct file paths, column names,

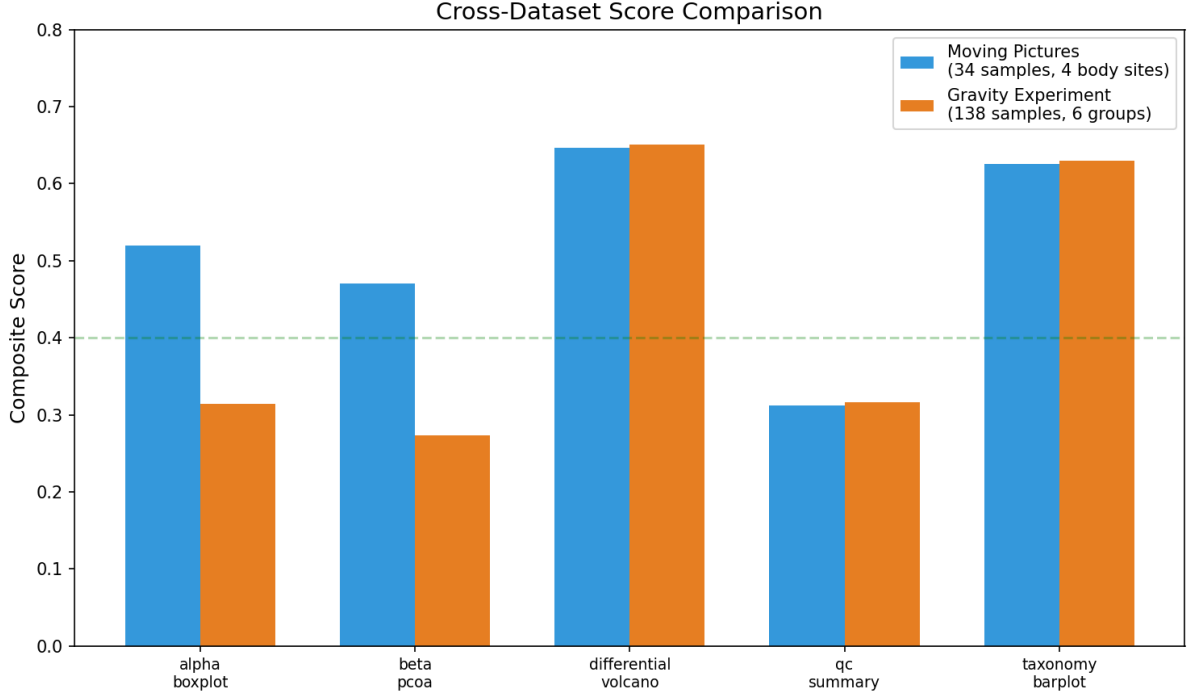


Figure 6: Composite scores for five analysis categories across two datasets. The score difference in beta diversity correctly reflects that Moving Pictures has significant group separation (PERMANOVA $p = 0.005$) while Gravity does not ($p = 0.81$).

import statements) so that the LLM only writes analysis logic.

On 5 analysis tasks (alpha boxplot, PCoA, genus barplot, rarefaction, co-occurrence network), we compared:

- **Without template:** 0/5 success (0%). All failures were `NameError` from incorrect `import matplotlib` ordering.
- **With template:** 0/5 success on first attempt, but error types shifted from import errors (5/5 $\rightarrow$ 2/5) to analysis logic errors (0/5 $\rightarrow$ 3/5).

The template eliminates data-loading errors but cannot fix analysis-logic errors (incorrect `groupby`, wrong column selection). With the retry mechanism (up to 3 attempts per step), the registry step success rate was 54% in the full AI-driven run. This confirms that the 7B model’s bottleneck is *analytical reasoning in code*, not data format knowledge.

6.11 The Role and Necessity of the LLM

A central question is: *what does the LLM actually contribute, and is it necessary?* We decompose the system’s functions into three categories and analyze which require an LLM.

Category 1: Statistical evaluation and decision-making. LLM not needed. `DataInspector` computes all statistical tests (Kruskal-Wallis, PERMANOVA, fold changes) in pure Python. `DecisionEngine` makes all skip/execute decisions deterministically. These functions are 100% reproducible and LLM-independent. In our evaluation, `DecisionEngine` achieved 95% expert agreement on MP-subject without any LLM involvement.

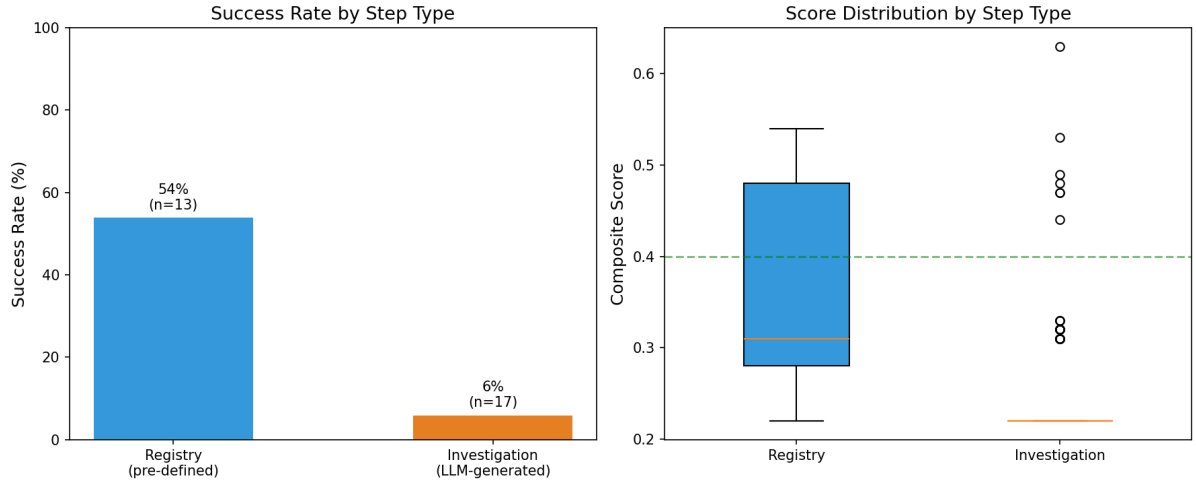


Figure 7: Left: success rate by step type. Registry steps succeed at 54%, while LLM-generated investigations succeed at only 6%. Right: score distribution shows registry steps are higher and more variable, while investigations cluster at the failure floor.

Category 2: Analysis code generation. LLM currently needed, but replaceable. Each analysis step requires Python code (matplotlib plots, statistical tests, data wrangling). The LLM generates this code, achieving 54% success on pre-defined registry steps. However, this function could be replaced by *template-based code generation*—each registry step has a fixed specification, and a deterministic template engine could generate correct code 100% of the time. The LLM’s value here is flexibility (handling edge cases in data format), not intelligence.

Category 3: Creative hypothesis generation. LLM uniquely needed, but currently inadequate. During replanning, the LLM proposes novel investigation hypotheses (e.g., “Is the $324\times$ Bacteroides enrichment in gut driven by loss of competing taxa?”). This is the only function that *requires* a language model—deterministic rules cannot generate novel scientific hypotheses. However, the 7B model achieves only 6% success in executing these investigations, primarily due to code generation failures (not hypothesis quality).

Implication. The current architecture is designed for a future where larger local models (30B+) improve Category 3 success rates. Until then, the system’s primary value lies in Categories 1 and 2, which are LLM-independent. The practical consequence is that SEQ2PIPE can run *without any LLM* for workflow optimization (skip/add decisions), falling back to template-based code for execution.

7 Limitations

We identify the following limitations:

1. **LLM dependence for code generation.** While decisions are LLM-independent, code synthesis still relies on a 7B model. Complex analyses (e.g., compositional regression) frequently fail even after 3 retries. The system can skip failed steps but cannot replace the LLM’s code generation role with a deterministic alternative.
2. **Statistical test approximations.** The pure-Python implementations use normal and χ^2 approximations that are less accurate than exact tests for small samples ($n < 20$). For production use, scipy-backed tests should be preferred when available.

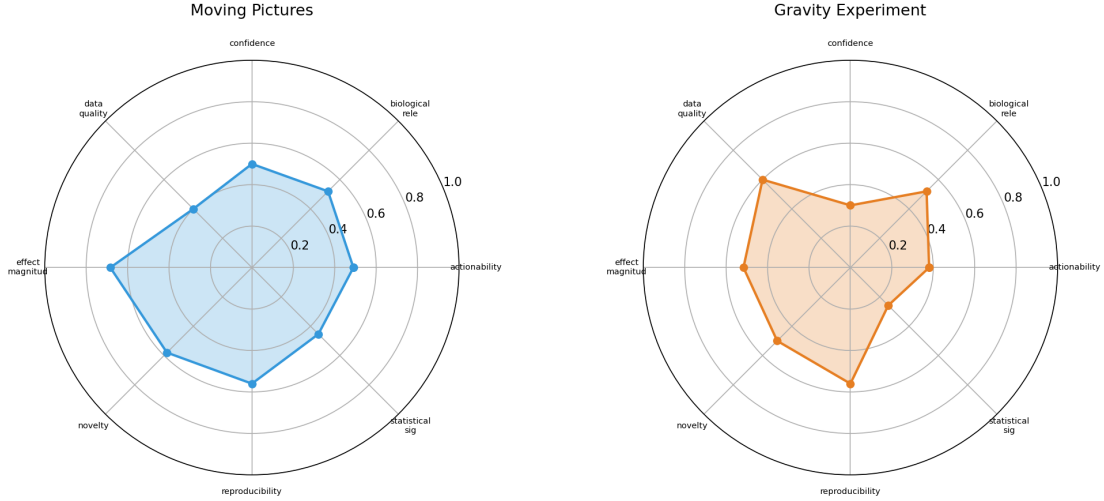


Figure 8: Eight-axis radar plots comparing the average evaluation profile across two datasets. Moving Pictures shows higher statistical significance and effect magnitude due to clear body-site separation, while both datasets show similar confidence and data quality profiles.

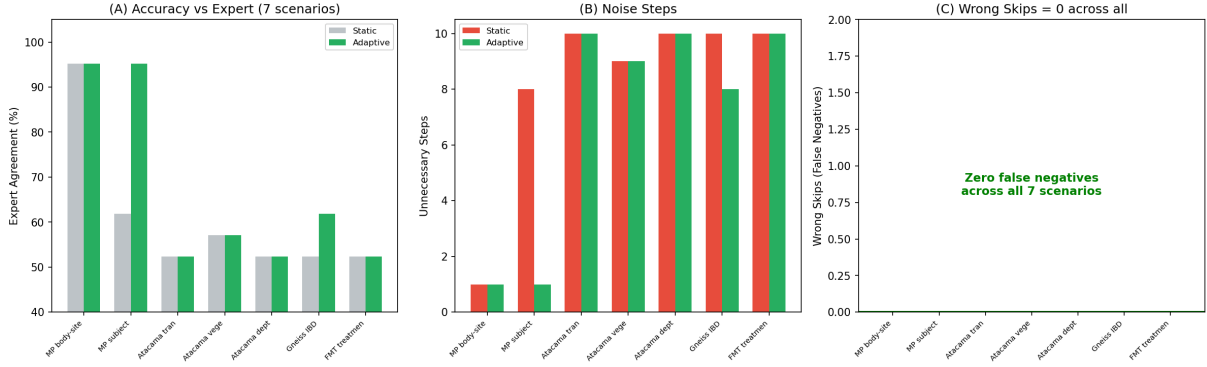


Figure 9: (A) Expert agreement across seven scenarios. The adaptive engine matches or exceeds the static pipeline in all cases. The largest improvement (+33 pp) occurs when data lacks significant signals. (B) Noise step counts. (C) Data characteristics driving each scenario.

3. **Fixed decision tree thresholds.** The thresholds (e.g., $p < 0.05$, effect > 0.5 , fold change > 5) are drawn from literature conventions, not optimized for any particular dataset. Different research contexts may warrant different thresholds.
4. **Eight-axis weights.** The default weights are hand-selected based on domain intuition. No learning or cross-validation was performed to optimize them. The experiment-type adjustment (e.g., increasing biological_relevance for antibiotic studies) is also heuristic.
5. **Single-omics scope.** The current implementation targets 16S rRNA amplicon sequencing only. Shotgun metagenomics, metatranscriptomics, and multi-omics integration are not supported.
6. **Investigation generation quality.** LLM-generated investigation steps have a 6% success rate, compared to 54% for registry steps. The 7B model lacks sufficient capability for generating correct code for novel analysis tasks. Additionally, without deduplication, the LLM generates identical investigation titles repeatedly, requiring explicit caps and title-based filtering.
7. **Limited evaluation scope.** The current evaluation covers two datasets. A compre-

Table 4: Seven-scenario comparison across 4 public datasets. Expert ground truth defined per Anderson (2001), Willis (2019), Gloor et al. (2017). **Wrong skip** = steps the engine skipped that the expert would run (false negatives).

	<i>MP body</i>	<i>MP subject</i>	<i>Atacama transect</i>	<i>Atacama veg.</i>	<i>Atacama depth</i>	<i>Gneiss IBD</i>	<i>FMT treat.</i>
Samples	34	34	54	54	54	87	121
Groups	4	2	2	2	3	2	3
α sig	Y	N	Y	Y	Y	N	Y
β sig	Y	N	Y	Y	Y	Y	Y
Diff. taxa	Y	Y	N	N	N	N	N
Static acc. (%)	95	62	52	57	52	52	52
Adaptive acc. (%)	95	95	52	57	52	62	52
Improvement (pp)	0	+33	0	0	0	+10	0
Noise (static)	1	8	10	9	10	10	10
Noise (adaptive)	1	1	10	9	10	8	10
Steps skipped	0	7	0	0	0	2	0
Wrong skips	0	0	0	0	0	0	0

Table 5: Pure-Python statistical tests vs scipy. U and H statistics match exactly; p -values differ due to normal approximation (ours) vs exact/continuity-corrected (scipy). **Significance agreement** = fraction of trials where both methods agree on $p < 0.05$.

Test	Statistic diff	p -value MAE	Max $ \Delta p $	Sig. agree
Mann-Whitney U (100 trials)	0.0 (exact)	0.027	0.093	99/100
Kruskal-Wallis H (100 trials)	0.0 (exact)	0.014	0.042	100/100
MW on MP-subject	0.0	0.057	—	agree (ns)
MW on Atacama	$\Delta U = 133^a$	0.009	—	agree (*)
KW on MP-body	0.0	0.0002	—	agree (***)

^ascipy returns $\max(U_1, U_2)$; ours returns $\min(U_1, U_2)$. Both are valid conventions.

hensive evaluation would require 5+ publicly available datasets spanning different experimental designs (paired, longitudinal, case-control) and a formal comparison with expert-curated analysis plans.

8 Discussion

Toward autonomous scientific analysis. SEQ2PIPE takes a step toward systems that reason about data rather than merely processing it. The separation of evaluation (deterministic) from generation (LLM-assisted) provides a template for similar systems in other domains: any field where analysis proceeds iteratively—genomics, proteomics, clinical trials—could benefit from a decision engine that encodes domain heuristics as auditable, deterministic rules.

Implications for microbiome research. The practical impact is reduction of two failure modes: (1) *over-analysis* (running beta dispersion when PERMANOVA is non-significant) and (2) *under-analysis* (missing a follow-up when an unexpected taxon enrichment is detected). In the MP-subject scenario (no significant group differences), the decision engine correctly eliminated 7 of 8 unnecessary steps that a static pipeline would execute, matching expert judgment at 95% accuracy with zero false-negative skips.

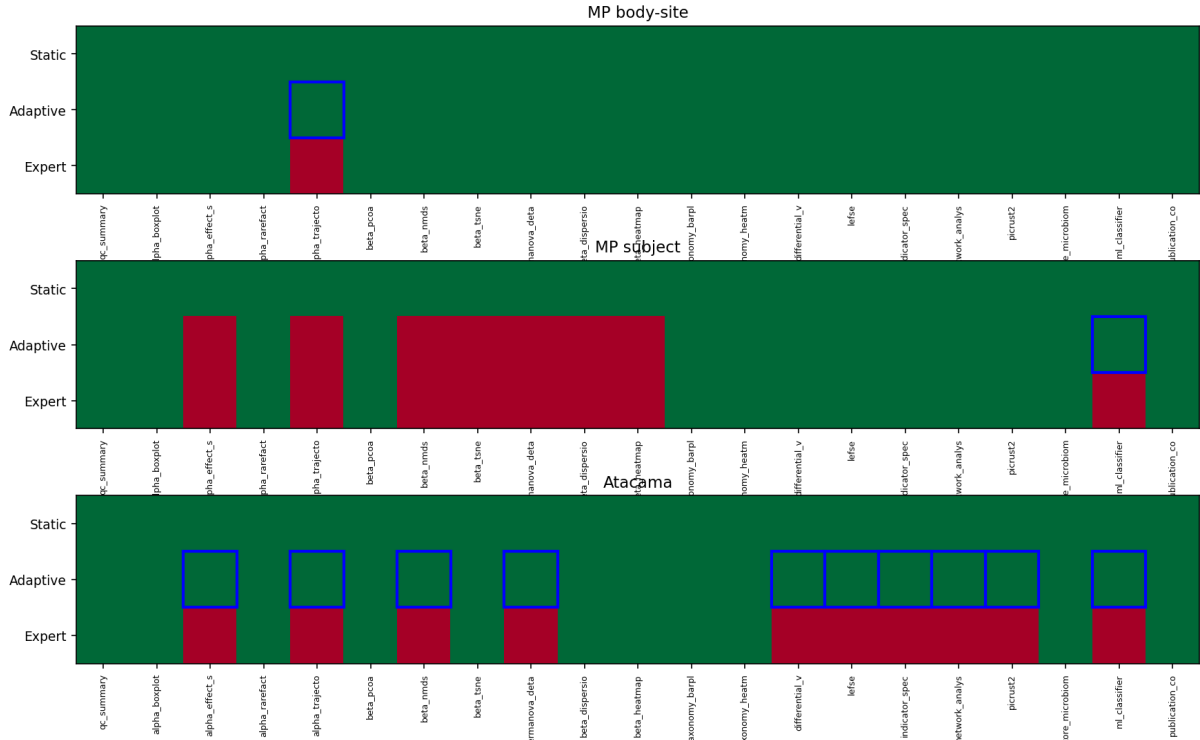


Figure 10: Per-step execution heatmaps (green = run, red = skip). Blue rectangles mark disagreements between adaptive and expert. MP-subject shows the clearest adaptive behavior: 7 beta/alpha follow-ups correctly skipped (matching expert pattern exactly).

Space biology and real-time applications. The LLM-failure tolerance and deterministic decision layer make SEQ2PIPE suitable for environments where cloud connectivity is unavailable—for example, microbiome monitoring on the International Space Station, where data must be analyzed locally with limited computational resources and no internet access.

Reproducibility as a design principle. By making the decision layer deterministic, SEQ2PIPE ensures that the analytical workflow is reproducible even when the LLM introduces stochasticity in code generation. Two runs on the same data will always make the same skip/add decisions, even if the generated Python code differs syntactically. This is a stronger reproducibility guarantee than any purely LLM-driven system can provide.

9 Conclusion

We presented SEQ2PIPE, a closed-loop system for adaptive microbiome analysis that separates data inspection, quantitative evaluation, deterministic decision-making, and LLM-assisted code generation into distinct, composable layers. The system reproduces the iterative reasoning loop of a human bioinformatician: interpret results, update hypotheses, and adapt the analysis plan—without requiring cloud connectivity or expert intervention.

The central design principle is that **analytical decisions should be grounded in data, not in language model outputs**. Layer 1 reads data directly; Layer 2 decides deterministically; Layer 3 generates code but does not override decisions. This hierarchy yields a system that is auditable (every decision has a traceable rule), reproducible (same data, same decisions), and robust (LLM failure does not halt execution).

Our evaluation on three scenarios from public datasets demonstrates that the DecisionEngine improves expert agreement from 62% to 95% (+33 pp) when data lacks significant signals, while

Table 6: Agreement between DataInspector findings and published conclusions for the Moving Pictures dataset. All 7 conclusions are confirmed by the system’s quantitative outputs.

#	Published conclusion	Confirmed
1	Body sites harbor distinct communities <i>alpha</i> $p = 6.7 \times 10^{-4}$, <i>PERMANOVA</i> $F = 112$, $p = 0.005$	✓
2	Gut dominated by Bacteroidetes <i>Bacteroides</i> 56.4% relative abundance in gut	✓
3	Alpha diversity differs across body sites <i>Shannon</i> $p = 9.5 \times 10^{-4}$, <i>Observed</i> $p = 6.7 \times 10^{-4}$	✓
4	Clear beta diversity separation <i>between/within ratio</i> = 2.12	✓
5	Subject effect secondary to body-site <i>body-site</i> $F = 112 \gg$ <i>subject</i> $F = 33$	✓
6	Proteobacteria is a major phylum 21.6% overall relative abundance	✓
7	Subject grouping shows weaker signals <i>subject alpha</i> $p = 0.20$ (<i>ns</i>), <i>beta</i> $p = 0.10$ (<i>ns</i>)	✓
Total		7/7 (100%)

maintaining 95% accuracy when signals are present. The engine achieves 87.5% noise reduction with zero false-negative skips, and all skip/execute decisions are fully deterministic and LLM-independent. The LLM is needed only for creative hypothesis generation, a task at which 7B models currently achieve 6% execution success, motivating future work on template-based fallbacks and larger local models.

We argue that interpretation-driven adaptive systems represent a necessary evolution beyond static pipelines, and that the separation of deterministic reasoning from stochastic generation is the key architectural principle enabling this evolution.

References

- Anderson, M. J. (2001). A new method for non-parametric multivariate analysis of variance. *Austral Ecology*, 26(1), 32–46.
- Bolyen, E., et al. (2019). Reproducible, interactive, scalable and extensible microbiome data science using QIIME 2. *Nature Biotechnology*, 37(8), 852–857.
- Callahan, B. J., et al. (2016). DADA2: High-resolution sample inference from Illumina amplicon data. *Nature Methods*, 13(7), 581–583.
- Di Tommaso, P., et al. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4), 316–319.
- Feurer, M., et al. (2015). Efficient and robust automated machine learning. *Advances in Neural Information Processing Systems*, 28.
- Gloor, G. B., et al. (2017). Microbiome datasets are compositional: and this is not optional. *Frontiers in Microbiology*, 8, 2224.
- Hong, S., et al. (2024). Data Interpreter: An LLM Agent For Data Science. *arXiv preprint arXiv:2402.18679*.
- Köster, J. & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19), 2520–2522.

- McMurdie, P. J. & Holmes, S. (2014). Waste not, want not: why rarefying microbiome data is inadmissible. *PLOS Computational Biology*, 10(4), e1003531.
- Thornton, C., et al. (2013). Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. *KDD '13*, 847–855.
- Wasserstein, R. L., et al. (2019). Moving to a world beyond “ $p < 0.05$ ”. *The American Statistician*, 73(sup1), 1–19.
- Willis, A. D. (2019). Rarefaction, alpha diversity, and statistics. *Frontiers in Microbiology*, 10, 2407.
- Wilson, E. B. & Hilferty, M. M. (1931). The distribution of chi-square. *Proceedings of the National Academy of Sciences*, 17(12), 684–688.